Professional Study Notes

Terraform Part 3

Terraform Language (HCL)

Beginner to Advanced • English with Kannada Explanations

AWS, Jenkins & EKS Examples

Part 3 — Terraform Language (HCL)

This module covers Terraform's HashiCorp Configuration Language (HCL), from basic syntax to advanced expressions, reusable data structures, functions, and scenario-based interview questions. It also revisits Terraform architecture, Infrastructure as Code (IaC), providers, resources, state, and the core workflow to connect the language concepts to real-world projects.

Learning objectives

By the end of this part, you should be able to:

- Read and write Terraform .tf configuration files.
- Understand blocks, arguments, expressions, and comments.
- Work confidently with strings, numbers, booleans, lists, tuples, sets, maps, and objects.
- Use Terraform functions and conditional expressions.
- Write maintainable AWS infrastructure configurations.
- Understand how HCL connects with Terraform providers, resources, dependency graphs, and state.
- Answer beginner, advanced, and production-scenario interview questions.

1. Terraform Language Fundamentals

Topic 1.1 — What is HCL?

1. Definition

HCL (HashiCorp Configuration Language) is a configuration language developed by HashiCorp. Terraform uses HCL to describe the infrastructure you want to create, update, or manage.

Terraform files usually have the .tf extension. Variable definition files commonly use .tfvars. Example:

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "app_logs" {
  bucket = "my-company-app-logs-example"
}
```

Here:

- provider identifies the AWS provider configuration.
- resource declares an AWS S3 bucket.
- region and bucket are arguments.
- "aws" and "app_logs" are local labels used to identify the provider configuration and resource.

> Note: The bucket name must be globally unique, and AWS permissions must allow the requested operation.

2. Purpose

HCL makes infrastructure definitions readable, structured, reusable, and reviewable.

> ಕನ್ನಡ HCL ಅಂದರೆ infrastructure ಅನ್ನು code ರೂಪದಲ್ಲಿ ವಿವರಿಸಲು ಬಳಸುವ language. AWS ನಲ್ಲಿ EC2, S3, VPC, EKS ಮುಂತಾದ resources ಹೇಗೆ ಬೇಕು ಎಂದು ನಾವು .tf files ನಲ್ಲಿ ಬರೆಯುತ್ತೇವೆ.

3. Working

Terraform reads HCL configuration and performs several important steps:

1. Parses and validates the configuration.
2. Loads the required provider plugins.
3. Reads current infrastructure information and state.
4. Creates an execution plan to compare desired and known actual infrastructure.
5. Uses providers to perform the planned API operations when you apply the plan.

4. Practical Example

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}

provider "aws" {
  region = var.aws_region
}

resource "aws_instance" "web" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t3.micro"

  tags = {
    Name = "terraform-web-server"
  }
}
```

This example demonstrates variables, provider configuration, a resource, and a map expression for tags. Replace the sample AMI with a valid AMI for your chosen region before applying.

5. Interview Questions & Answers

Q1. What is HCL in Terraform?

Answer: HCL is Terraform's primary configuration language. It describes infrastructure using blocks, arguments, expressions, variables, and structured values.

Q2. Is HCL a programming language?

Answer: HCL is primarily a declarative configuration language. It supports expressions, functions, conditionals, and iteration constructs, but it is not a general-purpose programming language like Python.

Q3. What is the difference between .tf and .tfvars?

Answer: .tf files define Terraform configuration. .tfvars files provide values for declared input variables.

Q4. Does Terraform execute HCL line by line?

Answer: No. Terraform parses the configuration, evaluates expressions, builds a dependency graph, and determines the operations required to reach the desired state.

Topic 1.2 — Infrastructure as Code (IaC)

1. Definition

Infrastructure as Code is the practice of defining and managing infrastructure using machine-readable configuration files rather than relying entirely on manual operations.

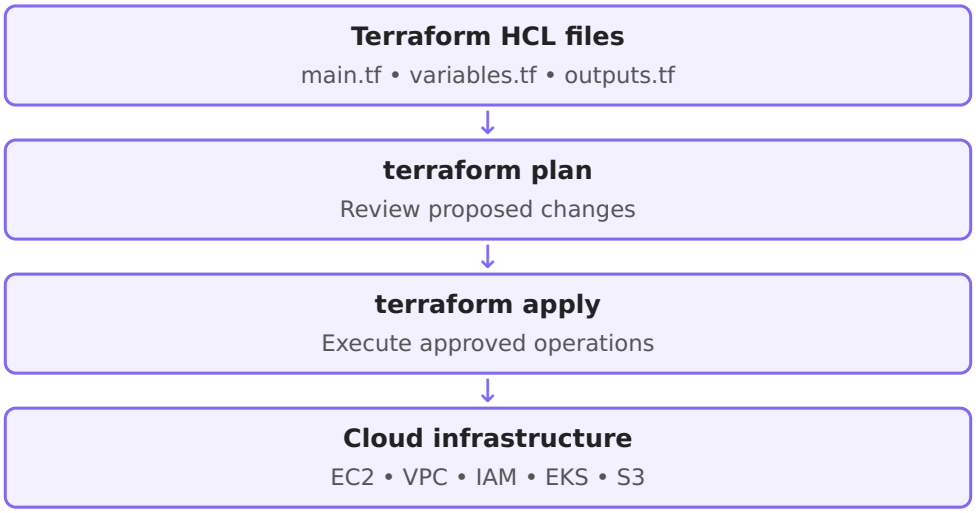
2. Purpose

- Automate infrastructure provisioning.
- Maintain consistent environments.
- Track changes using Git.

- Review infrastructure changes before applying them.
- Reproduce environments for development, testing, staging, and production.

ಕನ್ನಡ Console ನಲ್ಲಿ ಪ್ರತಿಯೊಂದು server ಅನ್ನು ಕೈಯಿಂದ create ಮಾಡುವ ಬದಲು, code ಮೂಲಕ infrastructure create ಮಾಡುವುದು IaC. ಅದರಿಂದ ಅದೇ configuration ಅನ್ನು ಮತ್ತೆ ಬಳಸಬಹುದು ಮತ್ತು Git ಮೂಲಕ changes track ಮಾಡಬಹುದು.

3. Working



In a CI/CD pipeline, Jenkins can run formatting, validation, security checks, planning, approval, and application steps.

4. Practical Example

Without IaC, an engineer manually creates an EC2 instance through the AWS Console. With IaC, the engineer declares:

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name = "application-server"
  }
}
```

The same configuration can be reviewed and applied consistently across environments, with appropriate variable values and permissions.

5. Interview Questions & Answers

Q1. What are the benefits of IaC?

Answer: Automation, repeatability, version control, auditability, reduced configuration drift, and faster environment provisioning.

Q2. How does Terraform support IaC?

Answer: Terraform uses configuration files to declare desired infrastructure, plans changes, and invokes provider APIs to apply them.

Q3. Is IaC completely error-free?

Answer: No. Incorrect configurations, security mistakes, provider limitations, and unexpected infrastructure changes can still occur. Validation, reviews, testing, and monitoring are important.

Topic 1.3 — Declarative vs. Imperative Approach

1. Definition

- Declarative: Specify the desired final state.
- Imperative: Specify the sequence of commands required to achieve a result.

2. Purpose

Declarative configuration lets Terraform determine which operations are needed, instead of requiring you to script every infrastructure operation.

> ಕನ್ನಡ Declarative ಅಂದರೆ “ನನಗೆ ಯಾವ infrastructure ಬೇಕು” ಎಂದು ಹೇಳುವುದು. Imperative ಅಂದರೆ “ಮೊದಲು ಇದನ್ನು ಮಾಡು, ನಂತರ ಅದನ್ನು ಮಾಡು” ಎಂದು step-by-step commands ಕೊಡುವುದು.

3. Working

Declarative	Imperative
Describes the desired result	Describes the steps
Terraform HCL is a common example	Shell scripts are a common example
Terraform calculates a plan	The script author controls command order
Can reconcile known differences	Requires explicit logic for many changes

4. Practical Example

Declarative Terraform:

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

Imperative shell example:

```
aws ec2 run-instances \
  --image-id "$AMI_ID" \
  --instance-type t3.micro
```

The AWS CLI command requests an instance launch. The Terraform declaration describes a managed resource whose desired configuration Terraform can compare against state and provider-reported infrastructure.

5. Interview Questions & Answers

Q1. Why is Terraform called declarative?

Answer: Because you describe the desired infrastructure state, and Terraform determines the operations required to move toward it.

Q2. Can Terraform perform imperative-looking operations?

Answer: Terraform supports provisioners and external automation integrations, but these should not replace native resource management when a provider resource is available.

Q3. Is a shell script always imperative?

Answer: Shell scripts commonly express imperative steps, although they can also call declarative tools such as Terraform or Kubernetes.

2. Terraform Architecture and Core Components

Topic 2.1 — Terraform Architecture

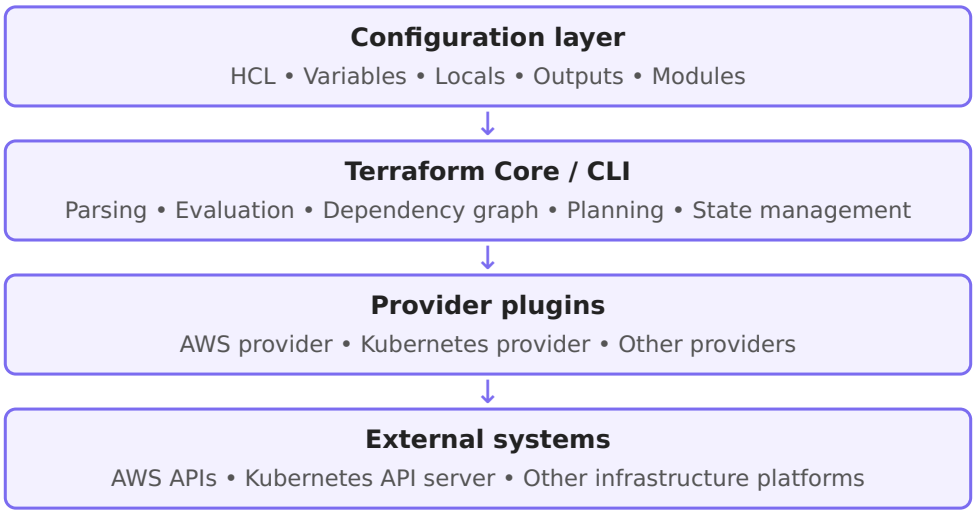
1. Definition

Terraform architecture describes how Terraform CLI, configuration files, providers, state, and remote APIs work together to manage infrastructure.

2. Purpose

Understanding the architecture helps you troubleshoot provider errors, state problems, authentication failures, dependency issues, and failed deployments.

3. Working



The state backend is a separate concern: it stores Terraform state locally or remotely. With a remote backend, state operations and locking depend on the backend's capabilities and configuration.

4. Practical Example

For your AWS EKS project:

1. eks-cluster.tf describes the EKS cluster.
2. provider.tf configures the AWS provider.
3. Terraform Core builds the plan and dependency graph.
4. The AWS provider communicates with AWS APIs.
5. Terraform records managed-resource information in state.
6. A configured S3 backend can store the state remotely and support team collaboration.

5. Interview Questions & Answers

Q1. What is Terraform Core?

Answer: Terraform Core processes configuration, evaluates expressions, constructs the dependency graph, generates plans, and coordinates resource operations with providers.

Q2. What is a provider plugin?

Answer: A provider is a plugin that enables Terraform to interact with a particular platform or API. The AWS provider manages AWS resources.

Q3. Does Terraform Core directly create EC2 instances?

Answer: Terraform Core coordinates the operation. The AWS provider performs the AWS API interactions required to manage the EC2 instance.

Q4. What happens if the AWS provider cannot authenticate?

Answer: Terraform cannot complete operations that require authenticated AWS API access. Check the credential chain, IAM permissions, AWS account, region, and role assumptions.

Topic 2.2 — Providers

1. Definition

A provider is a Terraform plugin that understands how to manage resources and retrieve data from a particular platform. Examples include the AWS, Kubernetes, Azure, and GitHub providers.

2. Purpose

Providers connect Terraform configurations to the APIs that manage real infrastructure.

> ಕನ್ನಡ Provider ಅಂದರೆ Terraform ಮತ್ತು AWS ನಡುವೆ ಸಂಪರ್ಕ ಕಲ್ಪಿಸುವ plugin. aws_instance ಮೂಲಕ EC2 manage ಮಾಡಲು AWS provider ಬೇಕು.

3. Working

A provider is declared using a required_providers block and configured with a provider block where necessary.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

- source identifies the provider's registry address.
- version constrains the provider version.
- region configures the AWS region.

The version constraint ~> 6.0 allows compatible versions within major version 6, subject to Terraform's version-constraint rules.

4. Practical Example

For your AWS project, run:

```
terraform init
terraform providers
terraform version
```

- terraform init installs required providers and initializes the working directory.
- terraform providers shows provider requirements in the configuration.
- terraform version reports Terraform and installed provider version information.

5. Interview Questions & Answers

Q1. Why do we need terraform init?

Answer: It initializes the working directory, installs required providers and modules, and initializes the configured backend.

Q2. What is .terraform.lock.hcl?

Answer: It records selected provider versions and package checksums so Terraform can select consistent provider packages across environments.

Q3. Is a provider the same as a resource?

Answer: No. The provider supplies resource types and API operations; a resource block declares a particular infrastructure object to manage.

Topic 2.3 — Resources

1. Definition

A resource block declares an infrastructure object that Terraform should manage.

2. Purpose

Resources let you create, update, and delete cloud infrastructure through configuration.

3. Working

General syntax:

```
resource "RESOURCE_TYPE" "LOCAL_NAME" {
  argument = value
}
```

Example:

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

Here:

- aws_instance is the resource type.
- web is the Terraform-local resource name.
- ami and instance_type are resource arguments.

The resource address is aws_instance.web. For a resource inside a module, the address may look like module.ec2.aws_instance.web.

4. Practical Example

```
resource "aws_security_group" "web" {
  name        = "web-server-sg"
  description = "Allow web traffic"

  ingress {
    description = "HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.10/32"]
  }
}
```

This example permits HTTPS from a single documentation-range IP address. Replace it with an appropriate trusted CIDR for a real environment.

5. Interview Questions & Answers

Q1. What is a resource address?

Answer: It is Terraform's identifier for a managed resource, such as aws_instance.web.

Q2. What happens when you remove a resource block?

Answer: Terraform generally plans to destroy the resource managed by that configuration, unless lifecycle rules or other circumstances change the result. Always inspect the plan before applying.

Q3. Can two resources depend on each other?

Answer: Terraform can represent dependencies, but a circular dependency cannot be resolved through its normal dependency graph and causes an error.

Topic 2.4 — Terraform State Overview

1. Definition

Terraform state is the record Terraform maintains to associate configuration resource addresses with real infrastructure objects and remember relevant resource attributes.

2. Purpose

State helps Terraform identify managed objects, calculate changes, track dependencies, and determine which infrastructure objects belong to the configuration.

> ಕನ್ನಡ State file ಅಂದರೆ Terraform manage ಮಾಡುತ್ತಿರುವ infrastructure ಬಗ್ಗೆ ಇರುವ record. Configuration ಏನು ಹೇಳುತ್ತದೆ ಮತ್ತು state/API ಏನು ಸೂಚಿಸುತ್ತವೆ ಎಂಬುದನ್ನು ಬಳಸಿಕೊಂಡು Terraform plan ತಯಾರಿಸುತ್ತದೆ.

3. Working

Common commands:

```
terraform state list
terraform state show aws_instance.web
terraform show
```

The default local state file is terraform.tfstate. A remote backend can store state centrally for team use. Important practices:

- Do not commit state files containing sensitive data to public Git repositories.
- Use access controls, encryption, and appropriate state locking.
- Back up state before performing advanced state operations.
- Prefer terraform state mv or terraform state rm only when you understand the consequences.

4. Practical Example

Suppose Terraform manages an EKS cluster. You rename a resource's local label from cluster to main_cluster. Terraform may interpret the new address as a different resource and propose replacement or recreation. Use a moved block when appropriate:

```
moved {
  from = aws_eks_cluster.cluster
  to   = aws_eks_cluster.main_cluster
}
```

This tells Terraform that the resource address changed without intending to replace the real cluster.

5. Interview Questions & Answers

Q1. Can Terraform work without state?

Answer: Terraform relies on state for normal management of tracked resources. State can be refreshed and reconciled with remote objects, but manually deleting it is not a safe way to reset a project.

Q2. What is state locking?

Answer: State locking prevents conflicting state-writing operations when supported and enabled by the backend.

Q3. Does state always contain the latest real-world values?

Answer: Not necessarily. External changes may occur between operations. Terraform refreshes relevant information during planning and applying, subject to configuration and command options.

3. HCL Blocks

Topic 3.1 — Blocks and Nested Blocks

1. Definition

A block is a structured section of HCL that groups related configuration. A block commonly has a type, optional labels, and a body enclosed in braces.

2. Purpose

Blocks organize configuration so that Terraform can identify providers, resources, variables, outputs, modules, and other configuration elements.

3. Working

General syntax:

```
BLOCK_TYPE "LABEL_1" "LABEL_2" {
  argument = value

  NESTED_BLOCK {
    nested_argument = value
  }
}
```

Examples of common blocks:

- terraform
- provider
- resource
- data
- variable
- output
- locals
- module

A nested block represents a structured configuration inside another block, such as an ingress block inside a security group resource. Whether a block or an object argument is required depends on the resource schema.

4. Practical Example

```
resource "aws_security_group" "app" {
  name = "app-security-group"

  ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.10/32"]
  }
}
```

resource is the outer block, and ingress is a nested block.

5. Interview Questions & Answers

Q1. What are the parts of a Terraform block?

Answer: The block type, any required labels, and a body containing arguments and/or nested blocks.

Q2. What is a nested block?

Answer: A block placed inside another block to represent structured configuration, such as a security group's ingress rule.

Q3. Is every block required to have two labels?

Answer: No. Different block types have different label requirements. For example, a locals block normally has no labels, while a resource block requires a resource type and local name.

4. Arguments and Expressions

Topic 4.1 — Arguments

1. Definition

An argument assigns a value to a named configuration field.

2. Purpose

Arguments configure how a provider resource, variable, module, or other Terraform construct should behave.

3. Working

Syntax:

```
argument_name = expression
```

The expression on the right-hand side may be a literal, a variable reference, a function call, a conditional, or a more complex value.

4. Practical Example

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = {
    Name        = "app-server"
    Environment = var.environment
  }
}
```

The arguments include ami, instance_type, and tags.

5. Interview Questions & Answers

Q1. What is the difference between a block and an argument?

Answer: A block groups configuration; an argument assigns a value to a named field within a block.

Q2. Can an argument contain a function call?

Answer: Yes. For example, name = lower(var.environment).

Q3. What happens if you provide an unsupported argument?

Answer: Terraform usually reports a configuration error during validation or planning because the provider schema does not recognize the argument.

Topic 4.2 — Expressions

1. Definition

An expression is a piece of HCL that produces a value.

2. Purpose

Expressions let you derive configuration values, combine inputs, select conditions, reference resources, and build dynamic infrastructure configurations.

3. Working

Common expression categories include:

- Literal values: "hello", 10, true
- References: var.region, aws_vpc.main.id
- Function calls: lower(var.environment)
- Operators: var.count + 1
- Conditionals: var.environment == "prod" ? "t3.small" : "t3.micro"
- Collection and object constructors: ["dev", "stage"], { Name = "app" }
- for expressions: [for name in var.names : upper(name)]

4. Practical Example

```
variable "environment" {
  type    = string
  default = "dev"
}

locals {
  instance_type = var.environment == "prod" ? "t3.small" : "t3.micro"
}
```

If environment is "prod", Terraform evaluates instance_type to "t3.small". Otherwise, it evaluates to "t3.micro".

5. Interview Questions & Answers

Q1. What does an HCL expression return?

Answer: A value, such as a string, number, boolean, collection, object, or resource-derived value.

Q2. What is a conditional expression?

Answer: It evaluates a condition and returns one of two values: condition ? true_value : false_value.

Q3. Can expressions reference resource attributes?

Answer: Yes. For example, aws_vpc.main.id references the ID attribute of a declared VPC resource. This can also establish an implicit dependency.

Topic 4.3 — Comments

1. Definition

Comments are notes in configuration files that Terraform ignores when interpreting the configuration.

2. Purpose

Comments document design decisions, explain non-obvious configuration, and help other engineers maintain infrastructure code.

3. Working

HCL supports three common comment forms:

```
# Single-line comment

// Also a single-line comment

/*
  Multi-line comment
*/
```

4. Practical Example

```
# Use the region approved for this environment.
provider "aws" {
  region = var.aws_region
}

/*
The production cluster uses multiple availability zones.
Review the subnet configuration before changing the cluster.
*/
```

5. Interview Questions & Answers

Q1. Does Terraform execute comments?

Answer: No. Comments are ignored by the configuration parser.

Q2. Can comments replace documentation?

Answer: Comments are useful, but reusable modules should also have clear variable descriptions, output descriptions, and README documentation.

Q3. Should you place secrets in comments?

Answer: No. Comments remain in source control and can expose credentials just as ordinary configuration can.

5. Terraform Data Types

Terraform's type system defines what kind of values variables, expressions, and resource arguments accept. Correct types help detect mistakes before infrastructure changes are applied.

Topic 5.1 — String

1. Definition

A string is a sequence of text characters enclosed in double quotes.

2. Purpose

Strings represent names, regions, AMI IDs, instance types, descriptions, and other textual configuration values.

3. Working

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}
```

Access the variable using var.aws_region.

4. Practical Example

```
provider "aws" {
  region = var.aws_region
}
```

ಕನ್ನಡ String ಅಂದರೆ text value. ಉದಾಹರಣೆಗೆ "us-east-1", "t3.micro".

5. Interview Questions & Answers

Q1. What is a string in Terraform?

Answer: A textual value represented using double quotes.

Q2. How do you combine strings?

Answer: Use interpolation or functions.

```
locals {
  resource_name = "${var.environment}-web-server"
}
```

Q3. Can a string contain a variable reference?

Answer: Yes. Terraform supports interpolation using ${...} inside quoted strings. Direct references such as name = var.environment are often simpler when no string combination is required.

Topic 5.2 — Number

1. Definition

A number represents a numeric value, including integers and decimal values.

2. Purpose

Numbers configure instance counts, storage sizes, ports, timeouts, and other numeric parameters.

3. Working

```
variable "desired_capacity" {
  type    = number
  default = 2
}
```

4. Practical Example

```
resource "aws_eks_node_group" "workers" {
  # Other required EKS node group arguments are omitted.

  scaling_config {
    desired_size = var.desired_capacity
    min_size     = 1
    max_size     = 3
  }
}
```

ಕನ್ನಡ Number type ಅನ್ನು count, size, port ಮುಂತಾದ numeric values ಗೆ ಬಳಸುತ್ತೇವೆ.

5. Interview Questions & Answers

Q1. Does Terraform have separate integer and float types?

Answer: Terraform uses a general number type, which supports whole numbers and decimal values.

Q2. What if a number variable receives invalid text?

Answer: Terraform reports a type-conversion or type-validation error if the value cannot be converted to the required type.

Topic 5.3 — Boolean

1. Definition

A boolean represents either true or false.

2. Purpose

Boolean values control optional features, enablement flags, and conditional infrastructure behavior.

3. Working

```
variable "enable_monitoring" {
  type    = bool
  default = true
}
```

4. Practical Example

```
resource "aws_cloudwatch_log_group" "app" {
  count = var.enable_monitoring ? 1 : 0

  name              = "/application/logs"
  retention_in_days = 30
}
```

If enable_monitoring is true, Terraform declares one log group. If false, it declares zero instances of this resource.

> ಕನ್ನಡ Boolean ಅಂದರೆ true ಅಥವಾ false. Feature enable ಅಥವಾ disable ಮಾಡಲು ಉಪಯೋಗಿಸುತ್ತೇವೆ.

5. Interview Questions & Answers

Q1. What are the valid boolean values?

Answer: true and false.

Q2. How do you conditionally create a resource?

Answer: Use count with a conditional expression or use for_each when managing a collection of independently identified resources.

Topic 5.4 — Null

1. Definition

null represents the absence of a value.

2. Purpose

It can indicate that an optional argument should be omitted or that an optional input has no value.

3. Working

```
variable "key_name" {
  type     = string
  default  = null
  nullable = true
}
```

A provider may interpret a null argument as omitted, depending on the resource schema and the argument's semantics.

4. Practical Example

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
  key_name      = var.key_name
}
```

If key_name is null, Terraform can omit the optional EC2 key-pair setting, subject to the provider schema.

5. Interview Questions & Answers

Q1. Is null the same as an empty string?

Answer: No. null means absence of a value; "" is a string containing zero characters.

Q2. When is null useful?

Answer: When a resource argument is optional and should only be configured when a value is provided.

6. Terraform Collection and Structural Types

Topic 6.1 — List

1. Definition

A list is an ordered collection of values that share a common element type.

2. Purpose

Lists represent ordered sets of values such as availability zones, subnet IDs, IP ranges, or security group identifiers.

3. Working

```
variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b"]
}
```

Access elements by index, starting at zero:

```
locals {
  first_zone = var.availability_zones[0]
}
```

4. Practical Example

```
variable "subnet_ids" {
  type    = list(string)
  default = ["subnet-aaa111", "subnet-bbb222"]
}

output "first_subnet" {
  value = var.subnet_ids[0]
}
```

These subnet IDs are illustrative; replace them with valid IDs when using them for AWS resources.

5. Interview Questions & Answers

Q1. Is a list ordered?

Answer: Yes. List element order is preserved.

Q2. How do you retrieve the first list element?

Answer: Use index [0].

Q3. How do you get the number of elements?

Answer: Use length(var.availability_zones).

Topic 6.2 — Tuple

1. Definition

A tuple is an ordered collection whose elements can have different types. Its element types and positions are specified in its type definition.

2. Purpose

Tuples represent fixed-position structures with potentially different value types.

3. Working

```
variable "server_details" {
  type    = tuple([string, number, bool])
  default = ["web-server", 2, true]
}
```

The first element is a string, the second is a number, and the third is a boolean.

4. Practical Example

```
locals {
  server_name = var.server_details[0]
  replica_count = var.server_details[1]
  enabled = var.server_details[2]
}
```

5. Interview Questions & Answers

Q1. How is a tuple different from a list?

Answer: A list has a common element type; a tuple defines the type of each element by position.

Q2. When might you use a tuple?

Answer: When a fixed sequence of values with different types is needed. For named fields, an object is often more readable.

Topic 6.3 — Set

1. Definition

A set is a collection of unique values of a common element type. It does not guarantee meaningful element ordering.

2. Purpose

Sets are useful when duplicate values must be removed and ordering does not matter.

3. Working

```
variable "allowed_cidrs" {
  type = set(string)

  default = [
    "203.0.113.10/32",
    "203.0.113.20/32"
  ]
}
```

Terraform can convert suitable collections into sets, removing duplicate elements.

4. Practical Example

```
locals {
  unique_zones = toset([
    "us-east-1a",
    "us-east-1b",
    "us-east-1a"
  ])
}
```

The resulting set contains two distinct availability zones.

5. Interview Questions & Answers

Q1. Can a set contain duplicates?

Answer: No. A set contains unique values.

Q2. Does a set preserve list ordering?

Answer: No. Do not depend on set iteration order.

Q3. When should you use a set instead of a list?

Answer: Use a set when uniqueness is important and order is irrelevant.

Topic 6.4 — Map

1. Definition

A map is a collection of values of the same type, indexed by string keys.

2. Purpose

Maps are commonly used for environment configuration, tags, region-specific settings, and mappings between names and values.

3. Working

```
variable "instance_types" {
  type = map(string)

  default = {
    dev  = "t3.micro"
    test = "t3.small"
    prod = "t3.medium"
  }
}
```

Retrieve a value using a key:

```
locals {
  selected_type = var.instance_types["dev"]
}
```

4. Practical Example

```
variable "common_tags" {
  type = map(string)

  default = {
    Project     = "EKS"
    Environment = "dev"
    ManagedBy   = "Terraform"
  }
}

resource "aws_s3_bucket" "logs" {
  bucket = "company-app-logs-example"

  tags = var.common_tags
}
```

5. Interview Questions & Answers

Q1. What is a map?

Answer: A key-value collection in which values share a common type.

Q2. How do you retrieve a map value?

Answer: Use var.map_name["key"] or an equivalent attribute reference when appropriate.

Q3. What happens if a map key does not exist?

Answer: Direct indexing produces an error. Use lookup() or try() when a fallback is appropriate.

Topic 6.5 — Object

1. Definition

An object is a structural type containing named attributes that can each have their own specified type.

2. Purpose

Objects group related configuration fields under a single variable or local value.

3. Working

```
variable "server_config" {
  type = object({
    name          = string
    instance_type = string
    volume_size   = number
    monitoring    = bool
  })

  default = {
    name          = "app-server"
    instance_type = "t3.micro"
    volume_size   = 20
    monitoring    = true
  }
}
```

4. Practical Example

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.server_config.instance_type

  root_block_device {
    volume_size = var.server_config.volume_size
  }

  monitoring = var.server_config.monitoring

  tags = {
    Name = var.server_config.name
  }
}
```

This shows how a structured variable keeps related settings together.

5. Interview Questions & Answers

Q1. How is an object different from a map?

Answer: An object can define different attribute types, while a map's values share one type.

Q2. Why are objects useful in modules?

Answer: They group related inputs and provide clear, structured type validation.

Q3. Can an object contain nested objects?

Answer: Yes. Object types can contain other objects, lists, maps, and other supported types.

Topic 6.6 — Map vs. Object vs. List vs. Tuple vs. Set

Type	Ordered?	Element values	Typical use
list(string)	Yes	Same type	Subnet IDs
tuple([...])	Yes	Different types allowed	Fixed-position data
set(string)	No meaningful order	Same type, unique	Unique names

Type	Ordered?	Element values	Typical use
map(string)	Keys identify values	Same value type	Environment-to-instance-type mapping
object({...})	Named attributes	Different attribute types allowed	Server configuration

Interview scenario: You need to store environment, instance_type, and volume_size, each with different types. Which type is best?

Answer: An object, because each attribute has a meaningful name and an explicitly declared type.

7. Terraform Functions

Topic 7.1 — What Are Terraform Functions?

1. Definition

Terraform functions are built-in operations that transform, combine, inspect, or select values in HCL expressions.

2. Purpose

Functions reduce repeated code, normalize inputs, manipulate collections, handle optional values, and construct resource names.

3. Working

General syntax:

```
function_name(argument1, argument2)
```

Examples:

```
lower("PRODUCTION")
upper("dev")
length(["a", "b", "c"])
```

Terraform evaluates these expressions to produce values.

4. Practical Example

```
variable "environment" {
  type    = string
  default = "DEV"
}

locals {
  environment_name = lower(var.environment)
  resource_name    = "app-${local.environment_name}"
}
```

The result is app-dev.

5. Interview Questions & Answers

Q1. What are Terraform functions?

Answer: Built-in functions used to transform and evaluate values in configuration expressions.

Q2. Can you create custom Terraform functions in HCL?

Answer: Terraform does not provide general user-defined HCL functions in the way general-purpose programming languages do. You can encapsulate reusable configuration in modules and use built-in functions.

Q3. Can functions call AWS APIs directly?

Answer: Ordinary Terraform language functions transform values; they are not general API clients. Providers and data sources are used for infrastructure API interactions.

Topic 7.2 — String Functions

1. Definition

String functions manipulate text values.

2. Purpose

They help standardize names, construct identifiers, and transform input values.

3. Working

Function	Purpose	Example result
lower()	Converts text to lowercase	lower("PROD") → "prod"
upper()	Converts text to uppercase	upper("dev") → "DEV"
title()	Capitalizes words	title("hello world") → "Hello World"
trimspace()	Removes surrounding whitespace	trimspace(" dev ") → "dev"
replace()	Replaces matching text	replace("app_dev", "_", "-") → "app-dev"
format()	Formats a string	format("%s-%s", "app", "dev") → "app-dev"
join()	Joins strings from a list	join("-", ["app", "dev"]) → "app-dev"
split()	Splits a string	split(",", "a,b") → ["a", "b"]

4. Practical Example

```
variable "project_name" {
  type    = string
  default = "EKS PROJECT"
}

variable "environment" {
  type    = string
  default = "DEV"
}

locals {
  normalized_project = replace(lower(var.project_name), " ", "-")
  resource_prefix    = format("%s-%s", local.normalized_project, lower(var.environment))
}
```

Result: eks-project-dev.

5. Interview Questions & Answers

Q1. How do you convert an environment name to lowercase?

Answer: lower(var.environment).

Q2. How do you combine multiple values with a separator?

Answer: Use join() for a list of strings or format() for a fixed format.

Q3. How do you replace underscores with hyphens?

Answer: replace(var.name, "_", "-").

Topic 7.3 — Numeric and Collection Functions

1. Definition

These functions perform numeric calculations and inspect or transform collections.

2. Purpose

They help determine resource counts, filter or transform values, remove duplicates, and calculate minimum or maximum values.

3. Working

Function	Purpose	Example
length()	Counts elements or characters	length(["a", "b"]) → 2
min()	Returns the minimum number	min(2, 5, 1) → 1
max()	Returns the maximum number	max(2, 5, 1) → 5
ceil()	Rounds upward	ceil(2.1) → 3
floor()	Rounds downward	floor(2.9) → 2
concat()	Combines lists	concat(["a"], ["b"]) → ["a", "b"]
distinct()	Removes duplicate list values	distinct(["a", "a", "b"]) → ["a", "b"]
flatten()	Flattens nested lists	flatten([["a"], ["b"]]) → ["a", "b"]
sort()	Sorts a list of strings	sort(["b", "a"]) → ["a", "b"]
reverse()	Reverses a sequence	reverse(["a", "b"]) → ["b", "a"]

4. Practical Example

```
variable "min_nodes" {
  type    = number
  default = 1
}

variable "requested_nodes" {
  type    = number
  default = 4
}

variable "max_nodes" {
  type    = number
  default = 3
}

locals {
  desired_nodes = min(
    max(var.requested_nodes, var.min_nodes),
    var.max_nodes
  )
}
```

The resulting desired node count is 3. This expression clamps the requested count between the configured minimum and maximum.

5. Interview Questions & Answers

Q1. What is the difference between distinct() and toset()?

Answer: distinct() returns a list with duplicate elements removed while retaining the first occurrence order. toset() converts a collection to a set, removing duplicates without preserving meaningful ordering.

Q2. When would you use flatten()?

Answer: When nested lists need to be converted into a single list, often when processing subnet or resource collections.

Q3. How do you prevent a desired count from exceeding the maximum?

Answer: You can use min() and max() together to constrain a value within a range, as long as the bounds are valid.

Topic 7.4 — Map, Object, and Optional-Value Functions

1. Definition

These functions retrieve values by key, merge maps or objects, and handle missing or invalid expressions.

2. Purpose

They simplify configuration overrides and reduce failures caused by optional data.

3. Working

Function	Purpose	Example
lookup()	Looks up a key with an optional default	lookup({dev="t3.micro"}, "prod", "t3.small")
merge()	Combines maps or objects	merge({env="dev"}, {team="ops"})
keys()	Returns map/object keys	keys({a=1, b=2})
values()	Returns map/object values	values({a=1, b=2})
try()	Returns the first expression that does not produce an error	try(var.config.name, "default")
can()	Tests whether an expression can be evaluated without an error	can(var.port > 0)

try() does not suppress every Terraform failure; it handles dynamic evaluation errors in its argument expressions.

4. Practical Example

```
variable "environment" {
  type    = string
  default = "dev"
}

locals {
  default_tags = {
    ManagedBy = "Terraform"
    Project   = "EKS"
  }

  environment_tags = {
    Environment = var.environment
  }

  common_tags = merge(
    local.default_tags,
    local.environment_tags
  )
}
```

Result:

```
{
  ManagedBy   = "Terraform"
  Project     = "EKS"
  Environment = "dev"
}
```

5. Interview Questions & Answers

Q1. What does lookup() do?

Answer: It retrieves a value from a map using a key and can return a fallback value when that key is missing.

Q2. What is the purpose of merge()?

Answer: It combines multiple maps or compatible objects. When keys overlap, the value from the later argument takes precedence.

Q3. What is the difference between try() and can()?

Answer: try() returns the first successfully evaluated expression; can() returns a boolean indicating whether an expression can be evaluated without an error.

Topic 7.5 — Type Conversion Functions

1. Definition

Type conversion functions explicitly convert values into supported Terraform types.

2. Purpose

They help normalize variable inputs and ensure that expressions produce values of the types required by resources or modules.

3. Working

Function	Purpose
tostring()	Converts a suitable value to a string
tonumber()	Converts a suitable value to a number
tobool()	Converts a suitable value to a boolean
tolist()	Converts a compatible collection to a list
toset()	Converts a compatible collection to a set
tomap()	Converts a compatible value to a map
tomap() / tolist()	Require compatible element/value types

4. Practical Example

```
variable "node_count_text" {
  type    = string
  default = "3"
}

locals {
  node_count = tonumber(var.node_count_text)
}
```

The local value node_count is numeric, making it suitable for calculations that require a number.

5. Interview Questions & Answers

Q1. Why use tonumber()?

Answer: To convert a numeric string to a number before performing numeric calculations.

Q2. Will every value convert successfully?

Answer: No. Invalid values cause conversion errors. Validate external inputs rather than assuming all strings contain valid numbers.

Topic 7.6 — File and Encoding Functions

1. Definition

Terraform provides functions to read file content and encode or decode structured data.

2. Purpose

They support user-data scripts, policy documents, configuration templates, and data serialization.

3. Working

Common functions include:

- file() — reads an existing file as text.
- filebase64() — reads a file and returns Base64-encoded content.
- jsonencode() — converts a Terraform value to JSON.
- jsondecode() — parses JSON text into a Terraform value.
- yamlencode() — converts a Terraform value to YAML.
- yamldecode() — parses YAML text into a Terraform value.
- templatefile() — renders a template file with supplied variables.

File-reading functions operate on files that exist when Terraform evaluates the configuration; they are not general mechanisms for reading files generated later during the same apply.

4. Practical Example

```
locals {
  policy_json = jsonencode({
    Version = "2012-10-17"

    Statement = [{
      Effect   = "Allow"
      Action   = ["s3:ListBucket"]
      Resource = "*"
    }]
  })
}
```

This creates a JSON-formatted policy string. In a real IAM policy, narrow Resource to the intended bucket or resources.

5. Interview Questions & Answers

Q1. Why use jsonencode() instead of manually writing JSON?

Answer: It converts Terraform values into valid JSON and reduces formatting and escaping mistakes.

Q2. What does templatefile() do?

Answer: It reads a template file, evaluates its template expressions using supplied variables, and returns the rendered text.

Q3. Can you use file() to read a file created by a resource during apply?

Answer: Not as a general workflow. file() reads files already present during configuration evaluation. Use an appropriate resource, data source, or external workflow when generated content is needed.

Topic 7.7 — Date and Time Functions

1. Definition

Date and time functions manipulate timestamps and format date-time values.

2. Purpose

They can help construct identifiers, evaluate time-based conditions, and format values used in configuration.

3. Working

Examples:

```
timestamp()
formatdate("YYYY-MM-DD", timestamp())
timeadd(timestamp(), "1h")
```

timestamp() returns the current UTC time as an RFC 3339 timestamp. Since it changes over time, placing it directly in a managed resource argument can cause repeated planned changes.

4. Practical Example

```
locals {
  current_date = formatdate("YYYY-MM-DD", timestamp())
}
```

This is useful for displaying a date, but avoid using continuously changing timestamps as stable resource names or tags unless recurring changes are intentional.

5. Interview Questions & Answers

Q1. Why can timestamp() cause a resource to change on every run?

Answer: Its result changes with time, so Terraform may continually detect a configuration difference.

Q2. Should timestamp() be used for stable production resource names?

Answer: Usually not. Prefer stable identifiers or a deliberate release/version input.

8. Variables, Locals, Outputs, and References

Topic 8.1 — Input Variables

1. Definition

Input variables allow values to be supplied to a Terraform configuration from outside the resource definitions.

2. Purpose

They make configurations reusable across environments without editing resource code for every change.

3. Working

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

Reference it with var.instance_type.

4. Practical Example

terraform.tfvars:

```
instance_type = "t3.small"
aws_region    = "us-east-1"
```

Run:

```
terraform plan
```

Terraform loads automatically named variable files such as terraform.tfvars and *.auto.tfvars. Other variable values can be supplied using -var, -var-file, environment variables, or interactive input.

5. Interview Questions & Answers

Q1. What happens if a required variable has no default and no supplied value?

Answer: Terraform prompts for the value in an interactive run or reports an error when it cannot obtain the value in a non-interactive context.

Q2. Is a .tfvars file automatically safe for Git?

Answer: No. It can contain sensitive values. Exclude secret-bearing variable files from source control and use an appropriate secrets-management mechanism.

Topic 8.2 — Local Values

1. Definition

Local values are named expressions defined inside a locals block.

2. Purpose

They reduce duplication, centralize calculations, and improve readability.

3. Working

```
locals {
  name_prefix = "${var.project_name}-${var.environment}"
}
```

Reference local values using local.name_prefix.

4. Practical Example

```
locals {
  common_tags = {
    Project     = var.project_name
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}
```

Use the same tags across resources:

```
resource "aws_s3_bucket" "logs" {
  bucket = var.bucket_name
  tags   = local.common_tags
}
```

5. Interview Questions & Answers

Q1. What is the difference between a variable and a local?

Answer: Variables accept external input; locals calculate or organize values inside the configuration.

Q2. Can locals reference variables?

Answer: Yes. Locals frequently use input variables and other local values.

Topic 8.3 — Outputs

1. Definition

Output values expose selected information from a Terraform configuration after evaluation or application.

2. Purpose

Outputs make useful values available to operators, CI/CD pipelines, and other Terraform configurations.

3. Working

```
output "instance_id" {
  description = "ID of the managed EC2 instance"
  value       = aws_instance.web.id
}
```

4. Practical Example

For an AWS deployment:

```
output "instance_public_ip" {
  value = aws_instance.web.public_ip
}
```

Read the output using:

```
terraform output instance_public_ip
```

5. Interview Questions & Answers

Q1. Can outputs expose sensitive information?

Answer: Yes. Marking an output as sensitive = true redacts it in many CLI displays, but it does not encrypt the value independently or guarantee that it is absent from state.

Q2. Can one Terraform configuration use another configuration's outputs?

Answer: Yes, through mechanisms such as terraform_remote_state, provided the relevant backend and state access are configured securely.

9. Operators and Conditional Expressions

Topic 9.1 — Operators

1. Definition

Operators perform arithmetic, comparisons, and logical operations on expressions.

2. Purpose

They allow Terraform to calculate values and select configuration based on conditions.

3. Working

Category	Operators	Example
Arithmetic	+, -, *, /, %	var.count + 1
Comparison	==, !=, >, <, >=, <=	var.env == "prod"
Logical	&&, ||, !	var.enabled && var.env == "prod"

4. Practical Example

```
locals {
  is_production = var.environment == "prod"
  create_backup = local.is_production && var.backup_enabled
}
```

5. Interview Questions & Answers

Q1. What is the difference between == and =?

Answer: = assigns a value to a named argument; == compares two values for equality.

Q2. What does && mean?

Answer: Logical AND. Both conditions must be true for the combined expression to evaluate to true.

Topic 9.2 — Conditional Expressions

1. Definition

A conditional expression selects one of two values based on a boolean condition.

2. Purpose

It supports environment-specific configuration and conditional resource settings.

3. Working

```
condition ? true_value : false_value
```

Both result expressions must be compatible with Terraform's type rules.

4. Practical Example

```
locals {
  instance_type = var.environment == "prod" ? "t3.small" : "t3.micro"
}
```

5. Interview Questions & Answers

Q1. How do you use a conditional expression in Terraform?

Answer: Write a condition followed by ? and the true result, then : and the false result.

Q2. Is a conditional the same as count?

Answer: No. A conditional chooses a value; count controls how many instances of a resource or module are declared. A conditional can be used to calculate count.

10. For Expressions, Dynamic Blocks, Count, and For Each

Topic 10.1 — For Expressions

1. Definition

A for expression transforms a collection into another list, tuple, map, or object.

2. Purpose

It avoids repetitive hardcoded values and allows values to be derived from existing collections.

> The source notes end here, partway through the "Working" section of Topic 10.1. The remaining topics (for expressions in detail, dynamic blocks, count, and for_each) were not included in the text provided.